2025 IEEE International Conference on Robotics and Automation (ICRA)
May 19-23, 2025. Atlanta, USA

JPG-SLAM: Joint Point-Gaussian Splatting Representation for Dense Dynamic SLAM

Kunrui Huang[1] , Wennan Yang[1], Pengwei Zhou[1], Li Li[1], and Jian Yao[1]

***Abstract*— This paper presents a simultaneous localization and mapping (SLAM) system to provide accurate pose estimation and dynamic scene reconstruction. Our approach proposes a Joint Point-Gaussian Splatting representation, which fully integrates the robustness of isotropic feature points in pose estimation and the flexibility of anisotropic 3D Gaussians in scene representation. This system does not need to suppress the anisotropic representation of Gaussian elements, which enables the mapping module to achieve finer scene representation with lower memory consumption. Additionally, in order to enhance the adaptability of the system in dynamic environments, we introduced a dynamic region recognition module and utilized 3D Gaussian Splatting and 4D Gaussian Splatting representations to represent static and dynamic regions respectively. Furthermore, we developed a local map management strategy for Gaussian Splatting mapping, effectively reducing the memory and computational resource usage in the mapping process. Experiments on public datasets demonstrate that our system achieves state-of-the-art tracking and mapping accuracy compared to existing baselines.**

Fig. 1. **Visualization and rendering results of Gaussian map.** Compared to SplaTAM [17], our method achieves more accurate rendering results. Additionally, our method does not suppress the anisotropy of Gaussian elements, allowing for a more compact and accurate scene representation with lower memory usage.

I. INTRODUCTION AND RELATED WORK

Reconstructing dynamic scene models with differentiable rendering capability holds significant importance in the field of robotics. For instance, these models can simulate realistic environmental interaction and generate lifelike rendering data. It enables a reduction in the gap between simulators and real-world robot applications while simultaneously lowering the costs associated with robot training and testing.

Traditional simultaneous localization and mapping (SLAM) systems, such as [1]–[4], have achieved accurate pose estimation and demonstrated computational efficiency. Additionally, widely adopted methods [5]–[8] have verified that integrating IMU information can enhance the accuracy and robustness of pose estimation. However, these methods have limitations in maintaining only a sparse point cloud map, which fails to capture the detailed geometric of the scene. Several dense SLAM methods use dense point clouds [9], TSDF [10], and voxel occupancy grids [11]–[13] as scene representations to capture the detailed geometric of the scene. However, these methods fail to provide differentiable rendering capability.

Recently, many SLAM methods with differentiable rendering capabilities have been proposed. SLAM approaches that utilize implicit neural representation [14]–[16] can provide realistic geometric and texture rendering results. However, methods based on implicit neural networks require computationally expensive per-pixel ray casting, limiting their application in real-time rendering. Some approaches aim to enhance computational performance by utilizing feature points for pose estimation [18] and explicit representation of neural radiance fields [15], [19]–[21]. Nevertheless, it remains a significant gap between these methods and the real-time application goal.

3D Gaussian Splatting (3DGS) [22] has emerged as a promising approach for scene representation, enabling real-time and high-quality rendering tasks. MonoGS [23] and SplaTAM [17] achieve both tracking and mapping based on the 3DGS representation, resulting in high-fidelity scene reconstruction. However, the anisotropic nature of 3DGS representation poses challenges for camera pose tracking, particularly in cases of rapid camera movements, as the scale of Gaussian elements along the viewing ray direction cannot be precisely constrained by keyframes in local sliding windows with the limited field of view. Despite attempts to address this issue through isotropic Gaussian representation [17] or cost function constraint on Gaussian element anisotropy [23], existing methods struggle to achieve a balance between accurate pose estimation and flexible map representation and lead to significant memory consumption as shown in Fig 1. Additionally, existing 3DGS SLAM methods are based on the assumption of a static scene and only rely on visual information for data association. Consequently,

[1] Kunrui Huang, Wennan Yang, Pengwei Zhou, Li Li, Jian Yao (Corresponding author) are with the School of Remote Sensing and Information Engineering, Wuhan University, P.R. China {jian.yao,kunruihuang}@whu.edu.cn

2025 IEEE International Conference on Robotics and Automation (ICRA) | 979-8-3315-4139-2/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICRA55743.2025.11127881

these methods are inadequate for application in prevalent robot scenarios, including those involving dynamic objects, weak textures, and rapid self-motion.

Recently, several SLAM systems specifically designed for dynamic environments have been proposed. To mitigate the impact of dynamic objects on pose estimation, several approaches employ multi-view geometry consistency combined with semantic [24] and object detection [25] techniques to remove visual observations of dynamic objects. In our approach, a dynamic regions detection module similar to DynaSLAM [24] is employed, but instead of using a two-stage semantic segmentation method [26], a single-stage instance segmentation method [27], is utilized. Several methods [28]–[32] have been developed to explore the reconstruction of dynamic scenes. However, these methods rely on known camera poses as input and lack online solutions. Moreover, existing methods model the entire scene as dynamic, which deviates from the real-world scenario where static scenes and dynamic objects coexist, resulting in unnecessary consumption of computational resources. StreetGS [33] focuses on road traffic scenes and distinguishes foreground vehicles from the static background. However, it is unable to model dynamic objects that do not possess rigid motion attributes.

To address these challenges, we propose JPG-SLAM, a dense SLAM system using Joint Point-Gaussian Splatting as scene representation. In our pose estimation process, we estimate the camera pose by optimizing a joint cost of the reprojection error of feature points, photometric and geometric rendering error of Gaussian Splatting. To improve the adaptability of the system in dynamic environments, we employ an efficient dynamic region recognition method based on both semantic and multi-view geometric information. We filter out observations from dynamic regions during pose estimation process and represent dynamic regions using 4D Gaussian Splatting [29] in the mapping process. Our approach requires fewer computational resources compared to existing 4DGS-based methods for dynamic scene reconstruction, as 3DGS representation requires fewer parameters compared to 4DGS representation for representing static regions. Finally, we propose a local map management strategy, which differentiates Gaussian Spaltting elements into those that need to be updated and those that are stable, and fix the stable elements during mapping iteration. This strategy effectively reducing the computational cost during scene reconstruction.

Our main contributions are summarized as follows:

1) We propose a Joint Point-Gaussian Spaltting scene representation and develop the first SLAM system based on Gaussian Spaltting representation that supports dynamic scene reconstruction.
2) We introduce a local map management strategy based on Gaussian Splatting mapping, effectively reducing the computational cost of scene reconstruction.
3) Extensive evaluations on a variety of datasets demonstrate competitive performance compared to baselines.

II. METHODOLOGY

A. Framework Overview

Fig. 2 provides an overview of our system. In this section, we initially introduce the scene representation that combines feature points and Gaussian Splatting (Section II-C). Subsequently, we proposed an efficient dynamic region recognition module that integrates both semantic and geometric information. IMU measurement, as well as feature points and Gaussian Splatting in the static regions are utilized to jointly optimize the camera state estimation in the tracking process. (Section II-B). Finally, we represent the static and dynamic regions using 3D and 4D Gaussians Splatting respectively , and optimize them with local map management strategies to achieve the accurate and efficient reconstruction of dynamic scenes. (Section II-D).

B. Joint Point-Gaussian Splatting Representation

In this section, our objective is to develop a dynamic scene representation to support the accurate and efficient reconstruction of dynamic scenes. Previous methods have struggled to strike a balance between accurate pose estimation and flexible scene representation, with relatively low training efficiency. To tackle these challenges, we introduce a novel scene representation called Joint Point-Gaussian Splatting, which utilizes feature point, 3D Gaussian [22] to represent static regions and utilizes 4D Gaussian [29] to represent dynamic regions, and perform joint rendering and updating. In the following, we will first introduce the static regions and dynamic regions representations separately, followed by the Joint Point-Gaussian Splatting representation and its implementation details.

Static regions representation. The static region model consists of a series of sparse feature points and 3D Gaussian elements. Due to the limited number of feature points, only a subset of the 3D Gaussian elements are associated with the feature points. Each 3D Gaussian element comprises a position vector $\boldsymbol{\mu}_s \in \mathbf{R}^3$ and a covariance matrix $\boldsymbol{\Sigma}_s$, which represents the mean and covariance of the 3D Gaussian element located in the world coordinate frame, respectively. In order to preserve the positive semidefinite property of the covariance matrix, it is decomposed into a rotation matrix $\mathbf{R}_s$ and a scale matrix $\mathbf{S}_s$ [22]. The recovery of the covariance matrix $\boldsymbol{\Sigma}_s$ is defined as:

$$\boldsymbol{\Sigma}_s = \mathbf{R}_s \mathbf{S}_s \mathbf{S}_s^T \mathbf{R}_s^T. \tag{1}$$

In addition to the position vector and covariance matrix, each Gaussian element includes opacity $\alpha_s \in \mathbf{R}$ and color information $\mathbf{c}_s \in \mathbf{R}^3$. For the 3D Gaussian elements associated with feature points, the center of the 3D Gaussian element and the 3D position of the feature point are considered as the same value, and observations from feature points and 3DGS were jointly optimized for this 3D position.

Dynamic regions representation. In our proposed scene representation, dynamic regions are expressed using a series of 4D Gaussian elements, which extends the 3D Gaussian by incorporating temporal information. Similar to the 3D

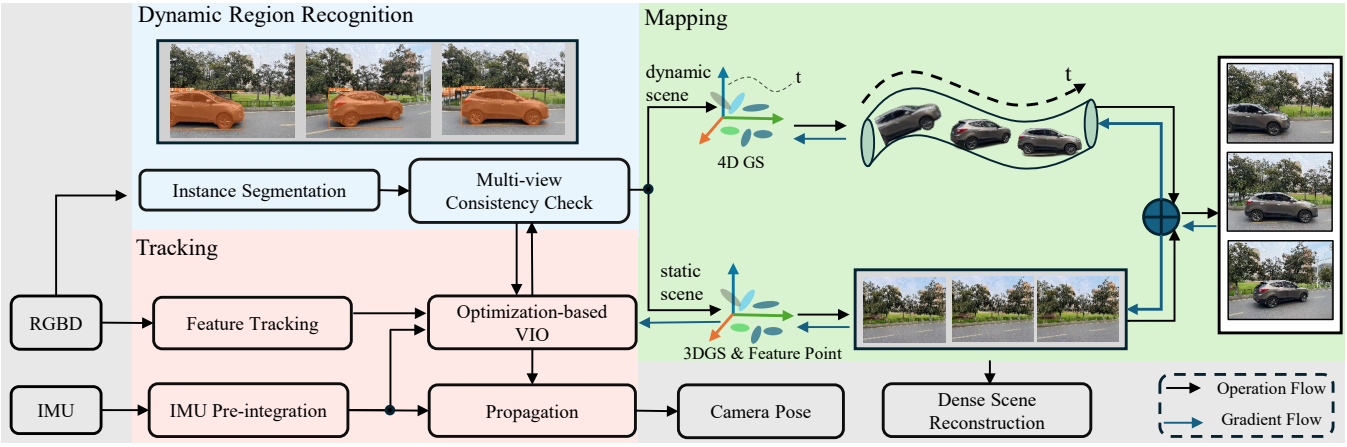

Fig. 2. **The framework of JPG-SLAM.** This framework consists of three main components. The dynamic region recognition process detects dynamic objects and performs instance segmentation in real-time. The tracking process detects and tracks feature points while performing IMU data preintegration between adjacent keyframes. The state optimization module combines feature tracking, IMU preintegration, and Gaussian Splatting rendering result to estimate camera pose. The Joint 3D-4D Gaussian Splitting Mapping process employs RGB-D data, keyframe pose, and dynamic region information to achieve a dense reconstruction of dynamic scenes with the Joint Point-Gaussian Splatting representation.

Gaussian, each 4D Gaussian comprises mean position vector $\boldsymbol{\mu}_d$, covariance matrix $\boldsymbol{\Sigma}_d$, color information $\mathbf{c}_d \in \mathbf{R}^3$ and opacity information $\alpha_d \in \mathbf{R}$.

Notably, the mean vector and covariance matrix are expanded to include dimensions related to time. The mean vector is represented as $\boldsymbol{\mu}_d = (\boldsymbol{\mu}_x, \boldsymbol{\mu}_y, \boldsymbol{\mu}_z, \boldsymbol{\mu}_t)$, while the covariance matrix decomposed following 3D Gaussian representation, with the scale matrix represented as $\mathbf{S} = \mathrm{diag}(\mathbf{s}_x, \mathbf{s}_y, \mathbf{s}_z, \mathbf{s}_t)$. In the context of 4D Euclidean space, a rotation can be decomposed into a pair of isotropic rotations, each represented by a quaternion [29].

Joint 3D-4D Gaussian Splatting representation. To generate the complete scene rendering, the rendering results from the static regions model and dynamic regions model are combined to obtain the final image. We combine the 3D Gaussian and 4D Gaussian elements to form a new Gaussian point cloud. We store 3D Gaussian and 4D Gaussian elements continuously in memory. It allows us to leverage the GPU thread ID to distinguish between 3D and 4D Gaussian elements while performing parallelized rendering and gradient backpropagation. Then, for each time moment t, the opacity $\alpha(t)$, position vector $\boldsymbol{\mu}_{xyz|t}$, and covariance $\boldsymbol{\Sigma}_{xyz|t}$ of all Gaussians are defined as follows:

$$\begin{aligned} \alpha(t) &= \alpha_s \cup \mathcal{N}\left(t; \boldsymbol{\mu}_4, \boldsymbol{\Sigma}_{4,4}\right) \alpha_d, \\ \boldsymbol{\mu}_{xyz|t} &= \boldsymbol{\mu}_s \cup \left(\boldsymbol{\mu}_{1:3} + \boldsymbol{\Sigma}_{1:3,4}\boldsymbol{\Sigma}_{4,4}^{-1}(t - \boldsymbol{\mu}_t)\right), \\ \boldsymbol{\Sigma}_{xyz|t} &= \boldsymbol{\Sigma}_s \cup \left(\boldsymbol{\Sigma}_{1:3,1:3} - \boldsymbol{\Sigma}_{1:3,4}\boldsymbol{\Sigma}_{4,4}^{-1}\boldsymbol{\Sigma}_{4,1:3}\right). \end{aligned} \tag{2}$$

Subsequently, using the camera's extrinsic parameters $\mathbf{W}$ and intrinsic parameters $\mathbf{K}$, we project Gaussian elements onto the pixel plane. The projection process of Gaussian elements is defined as follows:

$$\begin{aligned} \boldsymbol{\mu}' &= \mathbf{KW}\boldsymbol{\mu}, \\ \boldsymbol{\Sigma}' &= \mathbf{JW}\boldsymbol{\Sigma}\mathbf{W}^T\mathbf{J}^T. \end{aligned} \tag{3}$$

where $\mathbf{J}$ is the Jacobian matrix of $\mathbf{K}$. $\boldsymbol{\mu}'$ and $\boldsymbol{\Sigma}'$ are the position and covariance matrix in 2D pixel space, respectively.

Point-based α-blending is used to render the color image $\mathbf{I}_c$ and the depth image $\mathbf{I}_d$, which is defined as:

$$\begin{aligned} \mathbf{I}_c &= \sum_{i \in N} \mathbf{c}_i \alpha_i \prod_{j=1}^{i-1}(1 - \alpha_j), \\ \mathbf{I}_d &= \sum_{i \in N} \mathbf{d}_i \alpha_i \prod_{j=1}^{i-1}(1 - \alpha_j). \end{aligned} \tag{4}$$

C. Joint Point-Gaussian Splatting Tracking

Given a continuous stream of input images and IMU data, we initially employ the IMU data from neighboring image frames to estimate the initial position of feature points in the subsequent frame. Subsequently, the KLT sparse optical flow algorithm [34] is applied to track these feature points. To mitigate the interference of dynamic objects on pose estimation, we employ the single-stage semantic segmentation network YOLACT [27] for dynamic region recognition. Then in the backend optimization solver, the feature point tracking results, geometric and photometric errors of Gaussian Splatting rendering and pre-integrated IMU information from adjacent frames are utilized for pose optimization. During the optimization process, visual observations from dynamic regions are discarded based on instance segmentation results and predefined dynamic object categories. Additionally, robust kernel functions are employed for robust pose estimation. Finally, based on the pose estimation results, the dynamic object regions are further validated using a multi-view motion consistency method proposed in DynaSLAM [24]. The cost function for pose estimation is defined as:

$$\mathcal{L} = \mathcal{L}_{\text{repro}} + \mathcal{L}_{\text{color}} + \mathcal{L}_{\text{depth}} + \mathcal{L}_{\text{IMU}} \tag{5}$$

where $\mathcal{L}_{\text{repro}}$ represents the reprojection error of feature points, $\mathcal{L}_{\text{IMU}}$ represents the IMU preintegration error, and $\mathcal{L}_{\text{depth}}$ and $\mathcal{L}_{\text{color}}$ represents the geometric error and Photometric error of Gaussian Splatting rendering respectively.

IMU data provides high-frequency motion measurements, which yield accurate initial values for feature tracking and enhance the success rate of visual feature tracking. Additionally, the integration of IMU data can provide pose constraints for adjacent keyframes. This is particularly useful in scenarios involving rapid motion and insufficient visual features, as the fusion of IMU data significantly improves the accuracy and robustness of the pose estimation.

Feature points are isotropic and have high spatial resolution, which can provide accurate and reliable observations for pose estimation. we also incorporated the rendering error term of Gaussian Splatting rendering to better ensure consistency between pose estimation and Gaussian map. However, due to the anisotropic nature of Gaussian elements, we have applied a stricter outlier rejection strategy to the residual terms from the Gaussian elements. Furthermore, by filtering visual observations on dynamic objects using semantic information and combining robust kernel functions for pose estimation, the inlier ratio can be improved, and the influence of outliers on pose estimation can be reduced.

The multi-view geometric consistency verification module provides further verification of dynamic regions, thereby mitigating the impact of inaccurately defined motion states caused by predefined dynamic object categories. For instance, a chair, commonly considered a static object, may frequently be in motion during interactions with humans.

D. Joint Point-Gaussian Splatting Mapping

Given RGB and depth images, as well as the corresponding keyframe poses and dynamic regions information, the objective of this section is to design an efficient map management strategy and Gaussian map training strategy to achieve accurate and efficient reconstruction of dynamic scenes. In this section, we will first introduce the map management strategy and then discuss the design and implementation details of Gaussian map training.

Map Management strategy. Every time a keyframe is reached, additional Gaussian elements are introduced into the scene to capture the new visible scene and enhance the finer details. Similar to the SplaTAM [17], we determine the regions where new Gaussian elements need to be added based on the rendering error of the depth and color images. For the newly added feature points in the current keyframe, we initialize the position and covariance of the 3D Gaussian elements based on feature points' 3D position and reprojection error. For other regions without feature point coverage, we downsample the point cloud recovered from RGB-D data with a sampling factor of 0.05. We use 3D Gaussian and 4D Gaussian representations for new static and dynamic regions, respectively, based on the dynamic region recognition information.

We use the proportion of co-visible feature points to determine the similarity between two image frames and we select keyframes with a co-visibility ratio exceeding 0.2 and incorporate them into the local map keyframes. To mitigate the problem of map forgetting, two keyframes are randomly chosen from the database in each iteration to supplement the existing local map keyframes.

Additionally, we categorize Gaussian elements into three groups based on their visibility: observable to local keyframes, observable to historical keyframes, and not observable to all keyframes. We regard unobservable Gaussian elements as redundant and eliminate them. Moreover, we identify the Gaussian elements that are not observed by local map keyframes as stable and exclude them from the iterative optimization process. We only conduct iterative optimization on the Gaussian elements observed by the local map keyframes. This approach significantly reduces the number of parameters that require updating during iterations, leading to enhanced computational efficiency.

Gaussian Map training. We optimize the joint 3D-4D Gaussian map using the following loss function:

$$\mathcal{L} = \lambda_1 \mathcal{L}_{\text{depth}} + \lambda_2 \mathcal{L}_{\text{color}} + \lambda_3 \mathcal{L}_{\text{repro}}. \tag{6}$$

$\mathcal{L}_{\text{depth}}$ represents the difference between the rendered depth image and the ground truth depth image. Due to the inability to obtain accurate depth measurements in a few challenging scenarios using stereo cameras, we only utilize depth data within the operating range of stereo cameras and discard supervision signals with error values exceeding 10 times the median error. $\mathcal{L}_{\text{color}}$ represents the difference between the rendered color image and the ground truth RGB image. $\mathcal{L}_{\text{repro}}$ represents the projection error between the reprojection result of 3D Gaussian elements' center and the 2D feature point.

In dynamic scene mapping process, we use 3D Gaussian elements and 4D Gaussian elements as dense representations of static scenes and dynamic scenes respectively. Due to the fewer parameters of 3D Gaussian elements and the fact that most of the static areas in the scene are much larger than the dynamic areas, using 3D Gaussian representation can reduce the consumption of memory and computational resources. At the same time, we only update the elements in the local map during scene reconstruction, which further enhances the efficiency of scene reconstruction.

III. EXPERIMENTAL RESULTS

A. Experiment Setup

Dataset. TUMRGB-D dataset [35] and OpenLORIS dataset [36] are used for quantitative and qualitative comparison experiments and ablation studies.

Evaluation Metrics. To evaluate the accuracy of scene reconstruction, we employ the metrics of PSNR, SSIM, and LPIPS for rendered color images on 100 randomly sampled poses. In addition, we evaluate the precision of camera tracking using ATE RMSE on all keyframes.

Baseline. We compare our method with the SLAM system using implicit neural [16] and 3D Gaussian representation in terms of pose estimation and scene reconstruction [17], [23]. Due to the use of feature points and IMU data, our method is also compared with feature point-based visual and visual-inertial odometry [1], [5], [37]. As for the evaluation of dynamic scene reconstruction, we introduce 4DGS as an

additional baseline for comparison [29]. It is important to note that this comparison is unfair as the 4DGS method requires ground truth poses for scene initialization.

Implementation Details. We utilize the Adam optimizer [38] to train the Joint 3D-4D Gaussian map following the learning rate of 3DGS [22] and 4DGS [29]. In the VIO backend optimization module, we use 10 iterations for each keyframe. We update the Gaussian map for each keyframe using 60 iterations. Due to the large memory consumption of benchmark methods during training, we limit the length of each data sequence in the OpenLORIS dataset [36] to approximately 1000 frames. All the experiments are conducted on a desktop equipped with Intel Core i7 12900K 2.50GHz processor and a single NVIDIA RTX 3090 GPU.

TABLE I

POSE ESTIMATION RESULTS ON TUMRGB-D DATASET.

Methods	Metric	Avg.	Fr1-desk	Fr2-xyz	Fr3-rpy	Fr3-xyz
Point-SLAM [20]	ATE [m]↓	1.51	0.04	0.02	3.00	2.97
SplaTAM [17]	ATE [m]↓	0.22	0.03	0.02	0.39	0.43
MonoGS [23]	ATE [m]↓	0.38	**0.01**	0.02	0.73	0.76
Orbeez-SLAM [37]	ATE [m]↓	0.25	0.04	0.04	0.27	0.65
DynaSLAM [24]	ATE [m]↓	0.06	0.02	0.05	0.11	0.08
Ours(w/o IMU)	ATE [m]↓	**0.04**	0.02	**0.03**	**0.05**	**0.03**

TABLE II

POSE ESTIMATION RESULTS ON OPENLORIS DATASET

Methods	Metric	Avg.	Office	Home	Cafe	Market
Point-SLAM [20]	ATE [m]↓	3.70	0.15	2.61	6.73	5.29
SplaTAM [17]	ATE [m]↓	2.25	0.13	0.58	3.68	4.62
MonoGS [23]	ATE [m]↓	2.56	0.17	0.56	3.83	5.68
ORB-SLAM3 [1]	ATE [m]↓	0.47	0.23	0.41	0.46	0.79
VINS-Mono [5]	ATE [m]↓	0.42	0.19	0.34	0.48	0.68
Ours(w IMU)	ATE [m]↓	**0.29**	**0.11**	**0.25**	**0.37**	**0.41**

B. Comparisons with the State-of-the-art

Evaluation on TUMRGB-D dataset [35]. For the evaluation of pose estimation accuracy and robustness, we selected two data sequences of static scenes, fr1-desk and fr2-xyz, as well as two data sequences of dynamic scenes, fr3-rpy and fr3-xyz, for testing. Since the TUMRGB-D dataset does not provide IMU data, we deactivated the IMU measurement in the pose estimation section.

The evaluation results of pose estimation are shown in Table I. The pose estimation results show that our method achieves comparable results to the feature-based dynamic SLAM method and outperforms other methods. On data sequence of static scenes with slow motion changes, our method performs similarly to existing visual SLAM methods. In dynamic environments, existing SLAM methods with differentiable render ability depend on the assumption that the entire scene is static. The presence of dynamic objects significantly impacts the accuracy and robustness of pose estimation. The proposed method removes the visual observations within dynamic regions and utilize Gaussian Splatting rendering observation within static region, thus reducing the interference of dynamic objects on pose estimation and improving the accuracy and robustness of pose estimation.

TABLE III

SCENE RECONSTRUCTION RESULTS ON OPENLORIS DATASET

Methods	Metric	Avg.	Office	Home	Cafe	Market
Point-SLAM [20]	PSNR[dB]↑	16.92	17.68	16.25	16.76	16.97
	SSIM↑	0.56	0.63	0.51	0.54	0.56
	LPIPS↓	0.45	0.39	0.49	0.47	0.43
SplaTAM [17]	PSNR[dB]↑	19.49	22.98	21.73	16.69	16.57
	SSIM↑	0.71	0.86	0.82	0.62	0.53
	LPIPS↓	0.35	0.24	0.29	0.42	0.46
MonoGS [23]	PSNR[dB]↑	18.50	21.15	20.83	16.21	15.79
	SSIM↑	0.67	0.83	0.79	0.59	0.48
	LPIPS↓	0.38	0.25	0.31	0.43	0.54
Orbeez-SLAM [37]	PSNR[dB]↑	15.99	16.75	16.73	15.79	14.67
	SSIM↑	0.53	0.54	0.56	0.53	0.47
	LPIPS↓	0.49	0.47	0.45	0.51	0.54
4DGS [29] (offline method)	PSNR[dB]↑	**29.70**	30.16	**29.73**	**30.03**	**28.89**
	SSIM↑	**0.94**	0.94	**0.94**	**0.96**	0.90
	LPIPS↓	**0.15**	0.18	**0.17**	**0.09**	**0.14**
Ours	PSNR[dB]↑	29.06	**31.82**	28.33	28.39	27.68
	SSIM↑	**0.94**	**0.96**	0.92	0.95	**0.93**
	LPIPS↓	0.16	**0.15**	0.21	0.13	0.16

Evaluation on OpenLORIS Dataset [36]. The OpenLORIS dataset provides sensor and ground truth data for indoor ground robots. This dataset is more challenging as it contains scenes with weak textures, fast motion, and dynamic objects. We conducted experiments in four different scenarios, with the Office is a static scene, while the other three scenarios are dynamic scenes.

The evaluation results of pose estimation and scene reconstruction are presented in Table II and Table III, respectively. Compared to baseline SLAM methods, our approach achieves higher accuracy in pose estimation and scene reconstruction. By integrating feature points with isotropic properties , 3DGS rendering observation, and IMU information and effectively filtering out dynamic features, our method achieves accurate pose estimation results for all data sequences. Thanks to the excellent performance of feature point representation in pose estimation, we do not impose constraints on the anisotropic representation of Gaussian elements, which enables more precise rendering results. Furthermore, we use 3DGS and 4DGS to respectively represent static and dynamic scenes, which allows us to represent dynamic scenes accurately and efficiently. As shown in Fig. 3, our method achieves more accurate rendering results compared to existing SLAM methods. The rendering results are close to the offline method 4DGS [29]. Moreover, our method does not rely on ground truth pose data as input and requires fewer iterations.

C. Runtime Analysis

This section presents the evaluation of our proposed method and the baseline methods on the OpenLORIS dataset [36] in terms of running time and memory cost. The evaluation results are shown in Table IV.

In terms of pose estimation, our method integrates feature points, IMU, and 3DGS observation information. Thanks to the robustness of feature point expression in pose estimation, we achieved more accurate pose estimation in less than half of the existing 3DGS SLAM method. In the mapping module, our method has significant improvements compared to the 4DGS [29].The reason is that we employ

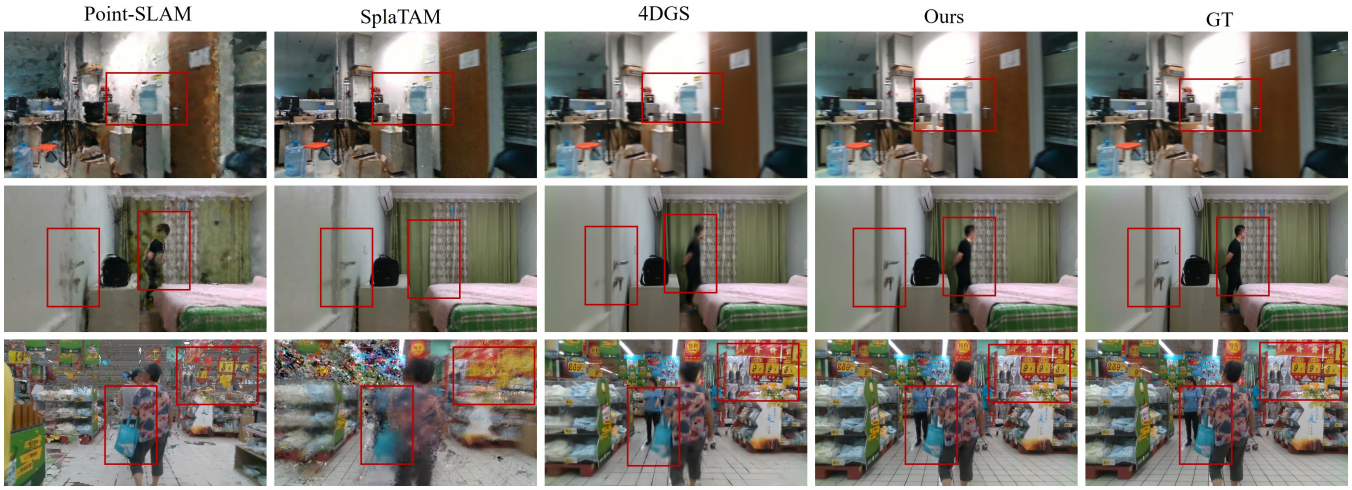

Fig. 3. **Qualitative evaluation results on the OpenLORIS dataset [36].** Point-SLAM [20] and SplaTAM [17] tend to produce blurry effects and cannot accurately reconstruct dynamic objects. Our method achieves accurate pose estimation and scene reconstruction in dynamic environments and achieves rendering accuracy close to the offline reconstruction method 4DGS [29].

TABLE IV

RUNTIME ON CAFE DATA SEQUENCE OF OPENLORIS DATASET

Methods	Tracking /Frame↓	Mapping /Iteration↓	Mapping /Frame↓	Map Memory↓	Rendering FPS↑
Point-SLAM [20]	0.84s	38ms	5.70s	**27.23MB**	1.25
SplaTAM [17]	1.04s	**29ms**	**1.74s**	247MB	**327**
MonoGS [23]	0.88s	31ms	1.86s	191MB	319
4DGS [29]	-	63ms	18.9s	78MB	123
Ours	**239ms**	37ms	2.22s	63MB	237

TABLE V

ABLATION STUDIES ON OPENLORIS DATASET

Methods	ATE↓	PSNR↓	Mapping /Frame↓	Map Memory↓
Ours w/o IMU	0.73m	25.48	2.08s	73.6MB
Ours w/o local map	0.33m	27.87	3.08s	69.3MB
Ours w/o 3DGS	0.53m	27.56	3.17s	75.3MB
Ours w/o 4DGS	0.38m	23.56	**1.83s**	**56.7MB**
Complete model	**0.29m**	**28.83**	2.17s	63.2MB

3D Gaussian representation for static regions, reducing the number of parameters that need to be updated through gradient backpropagation. Additionally, our proposed local map management strategy further reduces the number of parameters that need to be updated in each iteration. Because we did not suppress the anisotropic expression of Gaussian Splatting elements and expressed static and dynamic regions more accurately, compared to other 3DGS SLAM and 4DGS methods, we reduced the memory cost of map representation. Regarding real-time rendering, our method, along with all other methods using Gaussian representation, is capable of achieving a rendering frequency of over 100 FPS.

D. Ablations and Analysis

Ablation experiments were conducted on each module of our algorithm using the OpenLORIS dataset [36]. The results of the evaluations are presented in Table V.

Without incorporating IMU information, there is a significant decrease in the accuracy of pose estimation and scene reconstruction. The reason is that in scenarios with weak textures and fast motion, relying only on visual features can not provide sufficient and accurate constraints for pose estimation. Incorrect pose estimation subsequently affects the accuracy of scene reconstruction. Using the local map management strategy can reduce the computational time during the mapping process because the local map management strategy limits the number of map elements involved in updates, thus controlling the scale of the optimization problem. Without using 4DGS for representation, it is not possible to effectively represent dynamic regions, resulting in a decrease in rendering accuracy. On the other hand, using only 4DGS would lead to unnecessary waste of memory and computational resources. Additionally, since 4DGS elements cannot provide multi-view consistency constraints, it would also result in a decrease in pose estimation accuracy.

IV. CONCLUSIONS

In this paper, we propose a dense SLAM system called JPG-SLAM, which utilizes Joint Point-Gaussian Splatting as scene representation. By combining the accuracy of isotropic feature points in pose estimation and the flexibility of Joint 3D-4D Gaussian Splatting representation for dynamic scene reconstruction, we achieve more accurate pose estimation and dynamic scene reconstruction compared to existing 3DGS SLAM systems and neural implicit SLAM systems. Furthermore, we developed a local map management strategy for Gaussian elements, further improving the efficiency of scene reconstruction.

However, our method also has some known limitations. For instance, our system currently lacks a loop closure module, which inevitably leads to a certain amount of pose drift during long-term operation.

V. ACKNOWLEDGMENT

This work was partially supported by the National Natural Science Foundation of China (No.42271445) and the Shenzhen Science and Technology Program (JCYJ20230807090201003).

REFERENCES

[1] C. Campos, R. Elvira, J. J. G. Rodríguez, J. M. M. Montiel, and J. D. Tardós, "ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual–Inertial, and Multimap SLAM," *IEEE Transactions on Robotics*, vol. 37, no. 6, pp. 1874–1890, 2021.

[2] D. Caruso, J. Engel, and D. Cremers, "Large-scale direct slam for omnidirectional cameras," in *International Conference on Intelligent Robots and Systems (IROS)*, 2015.

[3] J. Engel, V. Koltun, and D. Cremers, "Direct Sparse Odometry," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 40, no. 3, pp. 611–625, 2018.

[4] C. Forster, M. Pizzoli, and D. Scaramuzza, "SVO: Fast Semi-Direct Monocular Visual Odometry," in *2014 IEEE International Conference on Robotics and Automation (ICRA)*, 2014, pp. 15–22.

[5] T. Qin, P. Li, and S. Shen, "VINS-Mono: A Robust and Versatile Monocular Visual-Inertial State Estimator," *IEEE Transactions on Robotics*, vol. 34, no. 4, pp. 1004–1020, 2018.

[6] P. Geneva, K. Eckenhoff, W. Lee, Y. Yang, and G. Huang, "OpenVINS: A Research Platform for Visual-Inertial Estimation," in *2020 IEEE International Conference on Robotics and Automation (ICRA)*, 2020, pp. 4666–4672.

[7] L. Xu, H. Yin, T. Shi, D. Jiang, and B. Huang, "EPLF-VINS: Real-Time Monocular Visual-Inertial SLAM With Efficient Point-Line Flow Features," *IEEE Robotics and Automation Letters*, vol. 8, no. 2, pp. 752–759, 2022.

[8] A. Rosinol, M. Abate, Y. Chang, and L. Carlone, "Kimera: an Open-Source Library for Real-Time Metric-Semantic Localization and Mapping," in *2020 IEEE International Conference on Robotics and Automation (ICRA)*, 2020, pp. 1689–1696.

[9] R. A. Newcombe, S. J. Lovegrove, and A. J. Davison, "DTAM: Dense Tracking and Mapping in Real-time," in *2011 IEEE/CVF International Conference on Computer Vision (ICCV)*, 2011, pp. 2320–2327.

[10] R. A. Newcombe, S. Izadi, O. Hilliges, D. Molyneaux, D. Kim, A. J. Davison, P. Kohi, J. Shotton, S. Hodges, and A. Fitzgibbon, "KinectFusion: Real-time Dense Surface Mapping and Tracking," in *2011 10th IEEE International Symposium on Mixed and Augmented Reality*, 2011, pp. 127–136.

[11] A. Hornung, K. M. Wurm, M. Bennewitz, C. Stachniss, and W. Burgard, "OctoMap: An Efficient Probabilistic 3D Mapping Framework Based on Octrees," *Autonomous robots*, vol. 34, pp. 189–206, 2013.

[12] W. Hess, D. Kohler, H. Rapp, and D. Andor, "Real-time loop closure in 2d lidar slam," in *2016 IEEE International Conference on Robotics and Automation (ICRA)*, 2016, pp. 1271–1278.

[13] L. Zhao, Y. Wang, and S. Huang, "Occupancy-SLAM: Simultaneously Optimizing Robot Poses and Continuous Occupancy Map," in *Proceedings of Robotics: Science and Systems*, New York City, NY, USA, June 2022.

[14] E. Sucar, S. Liu, J. Ortiz, and A. J. Davison, "iMAP: Implicit Mapping and Positioning in Real-time," in *2021 IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021, pp. 6229–6238.

[15] Z. Zhu, S. Peng, V. Larsson, W. Xu, H. Bao, Z. Cui, M. R. Oswald, and M. Pollefeys, "NICE-SLAM: Neural Implicit Scalable Encoding for SLAM," in *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 12 776–12 786.

[16] E. Sandström, Y. Li, L. Van Gool, and M. R. Oswald, "Point-SLAM: Dense Neural Point Cloud-based SLAM," in *2023 IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023, pp. 18 387–18 398.

[17] N. Keetha, J. Karhade, K. M. Jatavallabhula, G. Yang, S. Scherer, D. Ramanan, and J. Luiten, "SplaTAM: Splat, Track Map 3D Gaussians for Dense RGB-D SLAM," in *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.

[18] C.-M. e. a. Chung, "Orbeez-SLAM: A Real-time Monocular Visual SLAM with ORB Features and NeRF-realized Mapping," in *2023 IEEE International Conference on Robotics and Automation (ICRA)*, 2023, pp. 9400–9406.

[19] M. M. Johari, C. Carta, and F. Fleuret, "ESLAM: Efficient Dense SLAM System Based on Hybrid Representation of Signed Distance Fields," in *2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023, pp. 17 408–17 419.

[20] E. Sandström, Y. Li, L. Van Gool, and M. R. Oswald, "Point-SLAM: Dense Neural Point Cloud-based SLAM," in *2023 IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023, pp. 18 387–18 398.

[21] Z. Zhu, S. Peng, V. Larsson, Z. Cui, M. R. Oswald, A. Geiger, and M. Pollefeys, "NICER-SLAM: Neural Implicit Scene Encoding for RGB SLAM," in *International Conference on 3D Vision (3DV)*, 2024.

[22] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, "3D Gaussian Splatting for Real-Time Radiance Field Rendering," *ACM Transactions on Graphics*, vol. 42, no. 4, July 2023.

[23] H. Matsuki, R. Murai, P. H. J. Kelly, and A. J. Davison, "Gaussian Splatting SLAM," in *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.

[24] B. Bescos, J. M. Fácil, J. Civera, and J. Neira, "DynaSLAM: Tracking, Mapping and Inpainting in Dynamic Scenes," *IEEE Robotics and Automation Letters*, vol. 3, no. 4, pp. 4076–4083, 2018.

[25] J. Liu, X. Li, Y. Liu, and H. Chen, "RGB-D Inertial Odometry for a Resource-Restricted Robot in Dynamic Environments," *IEEE Robotics and Automation Letters*, vol. 7, no. 4, pp. 9573–9580, 2022.

[26] K. He, G. Gkioxari, P. Dollár, and R. Girshick, "Mask R-CNN," in *2017 IEEE/CVF International Conference on Computer Vision (ICCV)*, 2017, pp. 2961–2969.

[27] D. Bolya, C. Zhou, F. Xiao, and Y. J. Lee, "YOLACT: Real-time Instance Segmentation," in *2019 IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019, pp. 9157–9166.

[28] A. Pumarola, E. Corona, G. Pons-Moll, and F. Moreno-Noguer, "D-NeRF: Neural Radiance Fields for Dynamic Scenes," in *2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021, pp. 10 313–10 322.

[29] Z. Yang, H. Yang, Z. Pan, and L. Zhang, "Real-time Photorealistic Dynamic Scene Representation and Rendering with 4D Gaussian Splatting," in *International Conference on Learning Representations (ICLR)*, 2024.

[30] G. Wu, T. Yi, J. Fang, L. Xie, X. Zhang, W. Wei, W. Liu, Q. Tian, and W. Xinggang, "4D Gaussian Splatting for Real-Time Dynamic Scene Rendering," *arXiv preprint arXiv:2310.08528*, 2023.

[31] J. Luiten, G. Kopanas, B. Leibe, and D. Ramanan, "Dynamic 3d gaussians: Tracking by persistent dynamic view synthesis," *arXiv preprint arXiv:2308.09713*, 2023.

[32] L. Schmid, M. Abate, Y. Chang, and L. Carlone, "Khronos: A unified approach for spatio-temporal metric-semantic slam in dynamic environments," *arXiv preprint arXiv:2402.13817*, 2024.

[33] Y. Yan, H. Lin, C. Zhou, W. Wang, H. Sun, K. Zhan, X. Lang, X. Zhou, and S. Peng, "Street Gaussians for Modeling Dynamic Urban Scenes," *arXiv preprint arXiv:2401.01339*, 2024.

[34] B. D. Lucas and T. Kanade, "An Iterative Image Registration Technique with an Application to Stereo Vision," in *international joint conference on Artificial intelligence (IJCAI)*, 1981, pp. 674–679.

[35] J. Sturm, N. Engelhard, F. Endres, W. Burgard, and D. Cremers, "A Benchmark for the Evaluation of RGB-D SLAM Systems," in *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2012, pp. 573–580.

[36] X. Shi, D. Li, P. Zhao, Q. Tian, Y. Tian, Q. Long, C. Zhu, J. Song, F. Qiao, L. Song *et al.*, "Are We Ready for Service Robots? The OpenLORIS-Scene Datasets for Lifelong SLAM," in *2020 IEEE international conference on robotics and automation (ICRA)*, 2020, pp. 3139–3145.

[37] C.-M. Chung, Y.-C. Tseng, Y.-C. Hsu, X.-Q. Shi, Y.-H. Hua, J.-F. Yeh, W.-C. Chen, Y.-T. Chen, and W. H. Hsu, "Orbeez-slam: A real-time monocular visual slam with orb features and nerf-realized mapping," in *2023 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2023, pp. 9400–9406.

[38] D. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," in *International Conference on Learning Representations (ICLR)*, 2015.